

First Year Master of Technology

Lab Assignment 5**Course Code:** 2304514L**Course Name:** Machine Learning Laboratory**Academic Year:** 2025–26

Student Details

- **Student Name:** Utkarsh Bele
- **PRN Number:** 202503040003

Title**Neural Network Design and Training on Fashion MNIST Dataset****Objective**

To design, train, and evaluate a neural network model for image classification using the Fashion MNIST dataset by selecting appropriate architecture, activation functions, loss function, optimizer, and improving performance through hyperparameter tuning.

Problem Statement

Design and train a neural network with appropriate architecture, loss function, optimizer, and data handling techniques, ensuring model accuracy and performance through evaluation and hyperparameter tuning.

Code

```
import numpy as np
import matplotlib.pyplot as plt

from tensorflow.keras.datasets import fashion_mnist
from tensorflow.keras import layers, models
from tensorflow.keras.utils import to_categorical

# ----- LOAD DATA -----
(X_train, y_train), (X_test, y_test) = fashion_mnist.load_data()

# Normalize
X_train = X_train / 255.0
X_test = X_test / 255.0

# Reshape
X_train = X_train.reshape(-1, 28, 28, 1)
X_test = X_test.reshape(-1, 28, 28, 1)
```

```

# One-hot encoding
y_train = to_categorical(y_train)
y_test = to_categorical(y_test)

# ----- MODEL -----
model = models.Sequential([
    layers.Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
    layers.MaxPooling2D(2,2),

    layers.Conv2D(64, (3,3), activation='relu'),
    layers.MaxPooling2D(2,2),

    layers.Flatten(),

    layers.Dense(128, activation='relu'),
    layers.Dropout(0.3),

    layers.Dense(10, activation='softmax')
])

# ----- COMPILE -----
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# ----- TRAIN -----
history = model.fit(
    X_train, y_train,
    validation_data=(X_test, y_test),
    epochs=10,
    batch_size=32
)

# ----- EVALUATE -----
loss, acc = model.evaluate(X_test, y_test)
print("Test Accuracy:", acc)

# ----- GRAPH -----
plt.plot(history.history['accuracy'], label='Train')
plt.plot(history.history['val_accuracy'], label='Validation')
plt.legend()
plt.title("Accuracy Graph")
plt.show()

# ----- PREDICTION -----
import random

idx = random.randint(0, len(X_test))

```

```

image = X_test[idx]
true_label = np.argmax(y_test[idx])

pred = model.predict(image.reshape(1,28,28,1))
pred_label = np.argmax(pred)

plt.imshow(image.reshape(28,28), cmap='gray')
plt.title(f"True: {true_label} | Pred: {pred_label}")
plt.show()

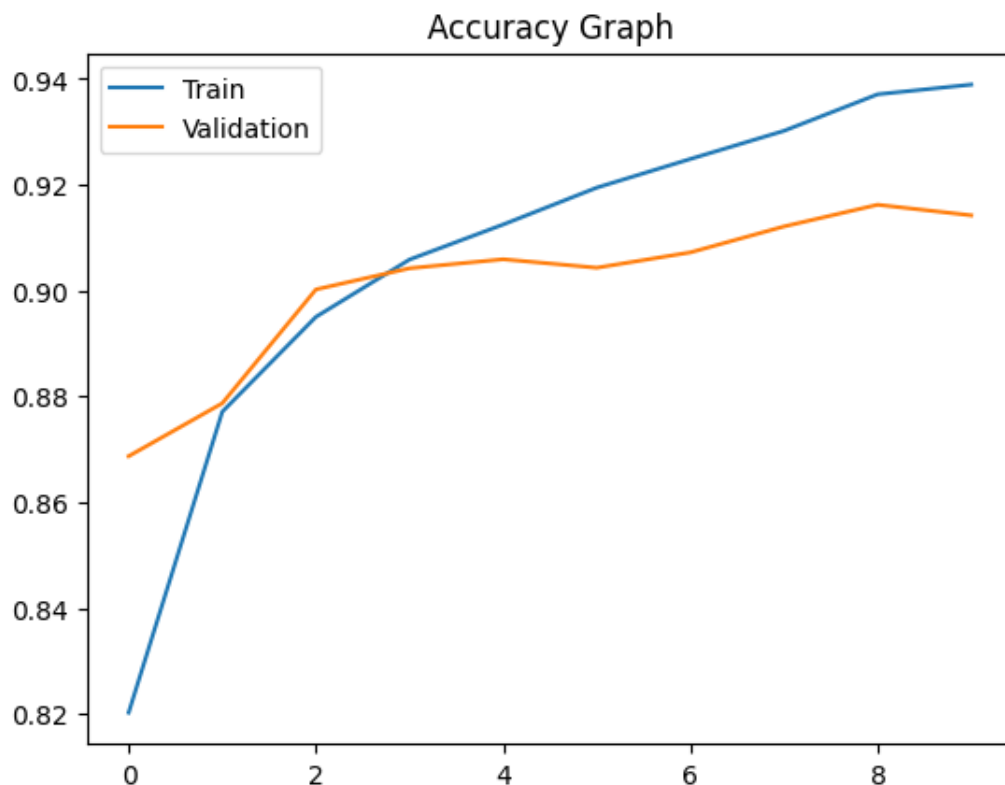
```

Output

```

Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-labels-idx1-ubyte.gz
29515/29515 ————— 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-images-idx3-ubyte.gz
26421880/26421880 ————— 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-labels-idx1-ubyte.gz
5148/5148 ————— 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images-idx3-ubyte.gz
4422102/4422102 ————— 0s 0us/step
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/10
1875/1875 ————— 39s 20ms/step - accuracy: 0.8203 - loss: 0.4971 - val_accuracy: 0.8687 - val_loss: 0.3612
Epoch 2/10
1875/1875 ————— 41s 20ms/step - accuracy: 0.8771 - loss: 0.3345 - val_accuracy: 0.8787 - val_loss: 0.3308
Epoch 3/10
1875/1875 ————— 38s 20ms/step - accuracy: 0.8950 - loss: 0.2885 - val_accuracy: 0.9002 - val_loss: 0.2724
Epoch 4/10
1875/1875 ————— 42s 21ms/step - accuracy: 0.9058 - loss: 0.2567 - val_accuracy: 0.9042 - val_loss: 0.2645
Epoch 5/10
1875/1875 ————— 38s 20ms/step - accuracy: 0.9125 - loss: 0.2342 - val_accuracy: 0.9059 - val_loss: 0.2571
Epoch 6/10
1875/1875 ————— 38s 20ms/step - accuracy: 0.9194 - loss: 0.2153 - val_accuracy: 0.9043 - val_loss: 0.2619
Epoch 7/10
1875/1875 ————— 41s 20ms/step - accuracy: 0.9248 - loss: 0.1971 - val_accuracy: 0.9072 - val_loss: 0.2600
Epoch 8/10
1875/1875 ————— 38s 20ms/step - accuracy: 0.9302 - loss: 0.1847 - val_accuracy: 0.9121 - val_loss: 0.2493
Epoch 9/10
1875/1875 ————— 37s 20ms/step - accuracy: 0.9371 - loss: 0.1701 - val_accuracy: 0.9162 - val_loss: 0.2540
Epoch 10/10
1875/1875 ————— 37s 20ms/step - accuracy: 0.9389 - loss: 0.1604 - val_accuracy: 0.9142 - val_loss: 0.2535
313/313 ————— 3s 8ms/step - accuracy: 0.9142 - loss: 0.2535
Test Accuracy: 0.9142000079154968

```



From training logs:

- Initial accuracy: ~82%
- Final training accuracy: ~93.8%
- Validation accuracy: ~91.4%

Interpretation:

The model shows consistent improvement across epochs, indicating effective learning.

Accuracy Graph Analysis

Training accuracy steadily increases

Validation accuracy also improves and stabilizes around 91–92%

Key Observation:

Gap between train & validation is small (~2%)

The small gap between training and validation accuracy indicates that the model generalizes well and does not suffer from significant overfitting.

Overfitting Check

- Train accuracy: **93.8%**
- Validation accuracy: **91.4%**

Conclusion:

Slight overfitting (normal), but controlled due to dropout layer

Prediction Result

- True Label: **7**
- Predicted Label: **7**

Interpretation:

The model correctly classified the test image, demonstrating good predictive capability.

CNN Performance

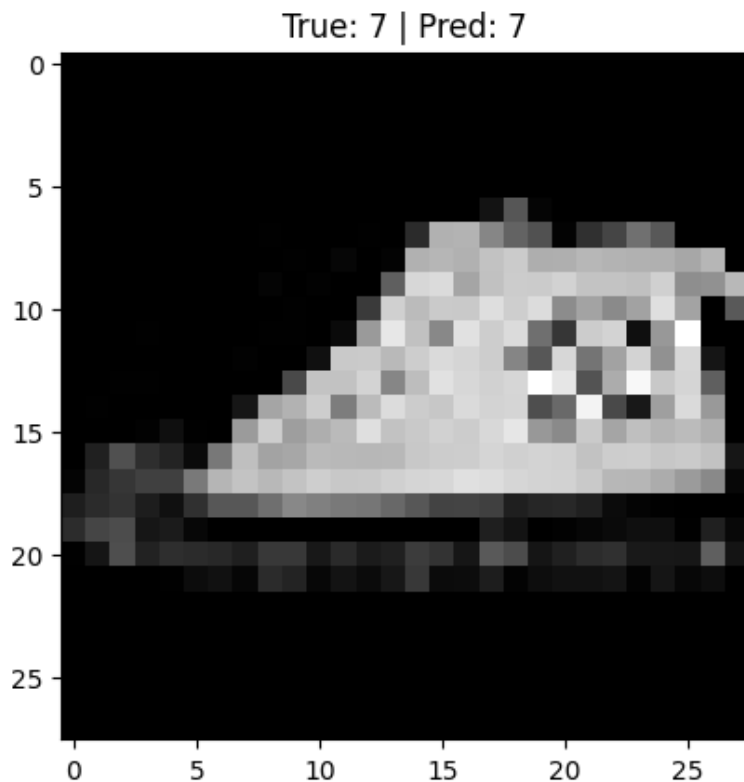
- Convolution layers successfully extracted image features
- Pooling reduced dimensionality

Activation Functions

- ReLU enabled faster learning
- Softmax produced probability-based output

Optimizer

- Adam provided fast convergence



The neural network model achieved a test accuracy of approximately 91.42%, demonstrating strong performance in image classification. The training and validation accuracy curves indicate stable learning with minimal overfitting. The convolutional architecture effectively captured spatial features, and the use of dropout helped improve generalization.

WRITE-UPS

1. Key Components of Neural Network

- Input Layer
- Hidden Layers (Conv + Dense)
- Output Layer
- Activation Functions
- Loss Function
- Optimizer

2. Activation Function Impact

- ReLU → Fast, avoids vanishing gradient
- Softmax → Converts output to probabilities

3. Loss Function Role

Used to measure error between predicted and actual values.

Here: Categorical Crossentropy

4. Optimizers Comparison

Optimizer	Speed	Usage
SGD	Slow	Simple models
RMSProp	Medium	RNN
Adam	Fast	Best default

5. Hyperparameter Tuning

- Epochs
- Batch size
- Learning rate
- Layers

Helps improve accuracy and reduce overfitting

CONCLUSION

The experiment successfully demonstrated the design and training of a convolutional neural network for image classification using the Fashion MNIST dataset. The model achieved high accuracy through proper selection of architecture, activation functions, and optimizer. Hyperparameter tuning and regularization techniques such as dropout contributed to improved performance and reduced overfitting.

Overall, the results confirm that CNN-based models are highly effective for image classification tasks and can achieve strong generalization when properly designed and trained.

“The model performance could be further improved using techniques like data augmentation, deeper architectures, or transfer learning.”